

# PROGRAMACIÓN II

## TRABAJO PRÁCTICO INTEGRADOR

PROFESORA: Naveda, Claudia

FECHA DE ENTREGA: 22/06/2026

INTEGRANTES: - Gagliardi, Franco  
- Cattáneo, Ciro  
- Quiroga Gabetta, Ramiro

## Food Store – Sistema de gestión de pedidos de comida (Consola + JDBC)

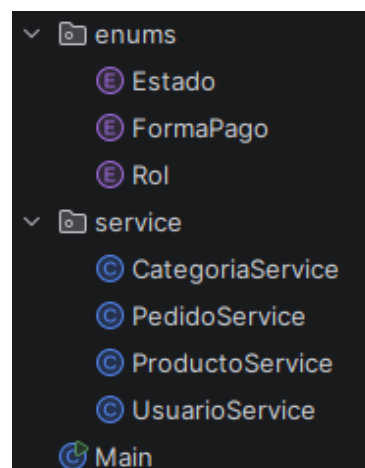
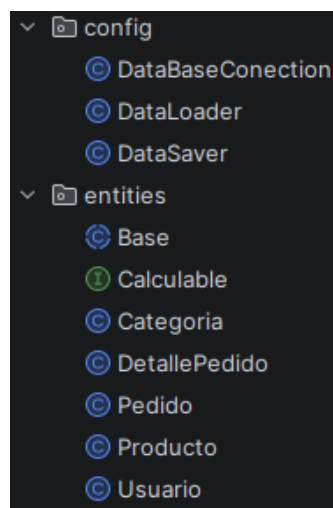
|                                                                                                                              |   |
|------------------------------------------------------------------------------------------------------------------------------|---|
|  MARCO TEÓRICO .....                      | 3 |
|  DECISIONES TÉCNICAS Y ARQUITECTURA ..... | 3 |
| Organización del Código y Distribución de Responsabilidades .....                                                            | 4 |
|  DIFICULTADES Y SOLUCIONES .....          | 4 |
|  CAPTURAS DE EJECUCIÓN DE CÓDIGO .....    | 5 |
|  BIBLIOGRAFÍA / WEBGRAFÍA .....           | 7 |
|  LINK AL REPOSITORIO .....                | 7 |

## 1. MARCO TEÓRICO (Tecnologías utilizadas y conceptos clave)

- [Java 25 + POO avanzada](#) → Se implementó una arquitectura basada en Herencia a partir de una clase abstracta común (Base) para heredar atributos transversales de auditoría (id, eliminado, createdAt). Se aplicó Encapsulamiento estricto mediante visibilidad privada y métodos de acceso, junto con la sobrescritura de métodos estructurales de la plataforma (toString y equals).
- [SQLite](#) → Motor de base de datos relacional ligero que almacena toda la estructura en un único archivo local. Prescinde de un servidor independiente, volviéndolo idóneo para aplicaciones de consola.
- [JDBC \(Java Database Connectivity\) y Colecciones](#) → API estándar de Java empleada para interactuar con la base de datos SQLite mediante instrucciones SQL nativas. Los datos recuperados mediante consultas son enviados dinámicamente hacia Colecciones en memoria cumpliendo el rol de estructuras operativas eficientes durante la ejecución de las reglas de negocio.
- [Interfaces y Abstracción](#) → Empleo de contratos lógicos como la interfaz Calculable para obligar la implementación del cálculo financiero automatizado (calcularTotal) en las capas de negocio.

## 2. DECISIONES TÉCNICAS Y ARQUITECTURA

Organizamos el árbol de directorios bajo el directorio raíz [src/main/java/](#) distribuyendo las responsabilidades mediante el siguiente esquema de paquetes, tal como se puede apreciar en el árbol de directorios del proyecto.



### Organización del Código y Distribución de Responsabilidades

Fragmentamos el dominio de la aplicación en cuatro capas lógicas independientes de responsabilidades bien definidas

- **Configuración e Infraestructura (config/)**: Centraliza la administración técnica de la persistencia. Mientras `DataBaseConection` gestiona el canal seguro con SQLite via JDBC, `DataLoader` y `DataSaver` moderan el traspaso de datos, encargándose de mapear los registros relacionales hacia las colecciones en memoria al iniciar la aplicación y volcar los cambios de regreso al finalizar la ejecución.
- **Modelo de Dominio y Abstracción (entities/)**: Contiene las clases estructurales derivadas del UML. Implementamos la herencia situando a `Base.java` como la clase abstracta de la cual heredan todas las entidades los atributos de auditoría (`id`, `eliminado`, `createdAt`). En este paquete incluimos la interfaz `Calculable.java`, logrando que `Pedido.java` la incorpore formalmente para resolver el método `calcularTotal()`.
- **Estructuras de Control (enums/)**: Aísla las constantes indispensables para delimitar los estados del negocio.
- **Lógica de Negocio y Casos de Uso (service/)**: Capa intermedia encargada de coordinar las operaciones CRUD y hacer respetar las reglas mínimas del sistema. Clases como `CategoriaService.java`, `ProductoService.java`, `UsuarioService.java` y `PedidoService.java` procesan las validaciones de control (como asegurar precios y stocks no negativos, o garantizar la unicidad del email) antes de impactar las colecciones.
- **Interfaz de Consola (Main.java)**: Gestiona exclusivamente los submenús interactivos mediante `Scanner`. Mantuvimos el diseño libre de reglas complejas; delega el flujo operativo a los servicios y encierra las llamadas en estructuras `try-catch` para capturar excepciones personalizadas.

## 3. Dificultades y soluciones

Dentro del marco del desarrollo del proyecto, no nos encontramos con una gran variedad de problemas, ya que el desarrollo tanto de la lógica del programa, siguiendo las reglas de negocio y las implementaciones de validaciones no nos representó una gran dificultad. No así con la implementación y posterior conexión del programa con la base de datos, en dicha fase del desarrollo de la aplicación surgieron múltiples problemas por errores de compilación o conexión con la base de datos. Ya configurada la conexión aparecieron algunos problemas por errores conceptuales del equipo, pero rápidamente pudimos llegar a una solución.

#### 4. Capturas de ejecución de código

```
BD creada correctamente
Datos cargados correctamente desde la BD.
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
```

*Menú principal en ejecución*

```
public class DataBaseConnection { 4 usages  Franco

    // Método centralizado: todo el que necesite una Connection pasa por acá,
    // así el PRAGMA foreign_keys siempre queda activado.
    public static Connection conectar(String url) throws SQLException { 3 usages
        Connection conn = DriverManager.getConnection(url);
        try (Statement stmt = conn.createStatement()) {
            stmt.execute(sql: "PRAGMA foreign_keys = ON;");
        }
        return conn;
    }
}
```

*Conexión a la base de datos dentro del package config*

```
// ---- CREATE (HU-CAT-02) ----
public Categoria crear(String nombre, String descripcion) { @ Franco
    if (nombre == null || nombre.isBlank()) {
        throw new IllegalArgumentException("El nombre no puede estar vacío");
    }
    boolean existe = categorias.stream()
        .anyMatch( Categoria c -> !c.isEliminado() && c.getNombre().equalsIgnoreCase(nombre));
    if (existe) {
        throw new IllegalArgumentException("Ya existe una categoría con ese nombre");
    }

    Categoria categoria = new Categoria(nombre, descripcion);
    categoria.setId(nextId++);
    categorias.add(categoria);
    return categoria;
}
```

*Creación de la HU-CAT-02*

```
public void setNombre(String nombre) { @ Ramiro Quiroga
    if (nombre == null || nombre.trim().isEmpty()) {
        System.out.println("Este campo es obligatorio, no puede estar vacío");
    } else if (!nombre.matches( regex: "[a-zA-ZáéíóúÁÉÍÓÚñÑ ]+")) {
        //Aseguro que el nombre solo contenga letras
        System.out.println("Ingrese un nombre válido");
    } else {
        this.nombre = nombre.trim();
    }
}
```

*Validación creada en el setter del atributo nombre, clase Usuario*

```
// Constructor para objetos NUEVOS (creados en memoria por el usuario).  
// Todavía no tienen id (lo asigna el Service al crear), arrancan  
// como no eliminados y con la fecha de creación actual.  
public Base() { 10 usages  🧑 Franco  
    this.eliminado = false;  
    this.createdAt = LocalDateTime.now();  
}  
  
// Constructor para objetos RECONSTRUIDOS desde la base de datos  
// (lo usa el DataLoader, que ya conoce el id, eliminado y createdAt reales).  
public Base(Long id, boolean eliminado, LocalDateTime createdAt) { 5 usages  🧑  
    this.id = id;  
    this.eliminado = eliminado;  
    this.createdAt = createdAt;  
}
```

*Constructores de la clase abstracta, Base*

## 5. Bibliografía / Webgrafía

- Aula virtual, campus TUP
- <https://www.youtube.com/@TodoCode>
- <https://www.youtube.com/@CharlyCimino>

## 6. Link al repositorio del proyecto

[https://github.com/franconicolasgagliardi01-spec/TPI\\_Programacion\\_2.git](https://github.com/franconicolasgagliardi01-spec/TPI_Programacion_2.git)